Bright Blue Store — Product Requirements & Engineering Spec

Project codename: `bbtc-store` **Owner:** Bright Blue Trading Corporation (BBTC), Ernakulam, Kerala **Document version:** 1.0 **Intended reader:** Claude Code (and any human developer joining the project)

---

0. How to use this document

This is both a PRD and a build manual. It is written to be handed to Claude Code as the source of truth for the entire project.

Instructions for Claude Code:

1. Read this document fully before writing any code.
2. Build in the phase order given in §14. Do not skip ahead. Each phase has acceptance criteria — verify them before moving on.
3. When a decision is ambiguous, prefer the option that keeps the admin panel simpler. The people using the admin are not technical.
4. Every currency value in this project is Indian Rupees, stored as **integers in paise**. Never use floats for money.
5. Ask before adding a dependency that is not listed in §5.
6. Keep a running `DECISIONS.md` at the repo root. Every time you make a judgement call not covered here, log it there in one line.

---

1. Overview

Bright Blue Trading Corporation manufactures and sells FMCG household products in India. The flagship product line is **MX3 dish wash liquid**, sold in six fragrance variants. The company also operates in car care, fabric care, and general home care.

The current site (brightblue.in) is a WordPress/WooCommerce install on a purchased theme. It has unfixed theme demo content, placeholder product images, and every product priced at ₹0.00. It is not usable as a store.

This project replaces it with a purpose-built ecommerce application: a fast, well-designed customer storefront, and a dedicated admin panel that non-technical staff can use to run products, pricing, offers, discounts, and combo packs without a developer.

Design reference: cleancult.com — specifically its category-first navigation, badge row of product claims, single-colour brand discipline, and clean product photography on flat backgrounds. We are taking the *structural and typographic discipline*, not the visual identity. Bright Blue gets its own identity (§11).

---

2. Business context

Facts the application must accommodate:

Item	Detail
Legal entities	Bright Blue Trading Corporation (BBTC), Bright Blue Manufacturing Pvt Ltd (BBM), Bright Blue Chemicals LLP (BBC)
Head office	Pullepady, Ernakulam, Kerala — 682035
Stock locations	Chengaram, Palakkad — 678681; Korad, Ozhur, Malappuram — 676307
Toll free	18001230621
Flagship SKU family	MX3 Dish Wash Liquid — Lemon, Peach, Blueberry, Apple, Grapes, Green Mango
Product categories	Dish Care, Home Care, Car Care, Fabric Care
Existing courier	DTDC
Currency	INR only
Primary market	Kerala, expanding pan-India

Commercial characteristics that shape the build:

- **Low unit price, low margin.** A bottle of dish wash is a low-ticket item. Average order value must be raised through multi-packs and combos — this is why combo support is a first-class feature, not an add-on.
- **COD is essential.** A large share of Indian FMCG orders outside metros are cash on delivery. COD must be supported from day one, with a per-pincode toggle and an optional COD fee.
- **B2B alongside B2C.** BBTC is a manufacturer with distributors and retailers. The system needs a wholesale price tier, even if it is not exposed publicly at launch.
- **GST compliance is mandatory.** HSN codes per product, correct CGST/SGST vs IGST split, and a downloadable tax invoice per order.

---

3. Goals and non-goals

Goals

- **G1.** A storefront that loads in under 2.5s LCP on a mid-range Android phone on 4G.
- **G2.** An admin panel where a non-technical staff member can add a product, set a price, upload photos, build a combo, and launch a discount — with no developer involvement.
- **G3.** Working checkout with online payment (Razorpay) and COD.
- **G4.** GST-compliant invoicing.
- **G5.** A design that is clearly better than every competitor in Indian home care.

Non-goals (explicitly out of scope for v1)

- Subscriptions / recurring orders
- Multi-language (Malayalam) UI — the schema will allow for it later, the UI will not ship it
- Mobile apps
- Marketplace integrations (Amazon, Flipkart, Blinkit sync)
- A full CMS. Marketing content is limited to a small set of editable homepage blocks.
- Loyalty points, referrals, gift cards
- Multi-currency

---

4. Users and roles

Role	Who	Capabilities
`GUEST`	Anonymous shopper	Browse, add to cart, checkout as guest
`CUSTOMER`	Registered shopper	All of guest, plus saved addresses, order history, reorder
`STAFF`	Warehouse / branch staff	View orders, update fulfilment status, print invoices. Cannot change prices or products.
`MANAGER`	Marketing / ops lead	All of staff, plus create and edit products, combos, discounts, homepage blocks
`ADMIN`	Owner / CMD	Everything, plus user management, settings, and access to financial reports

Role checks are enforced **server-side on every mutation**. Never rely on hiding a UI element as an access control.

---

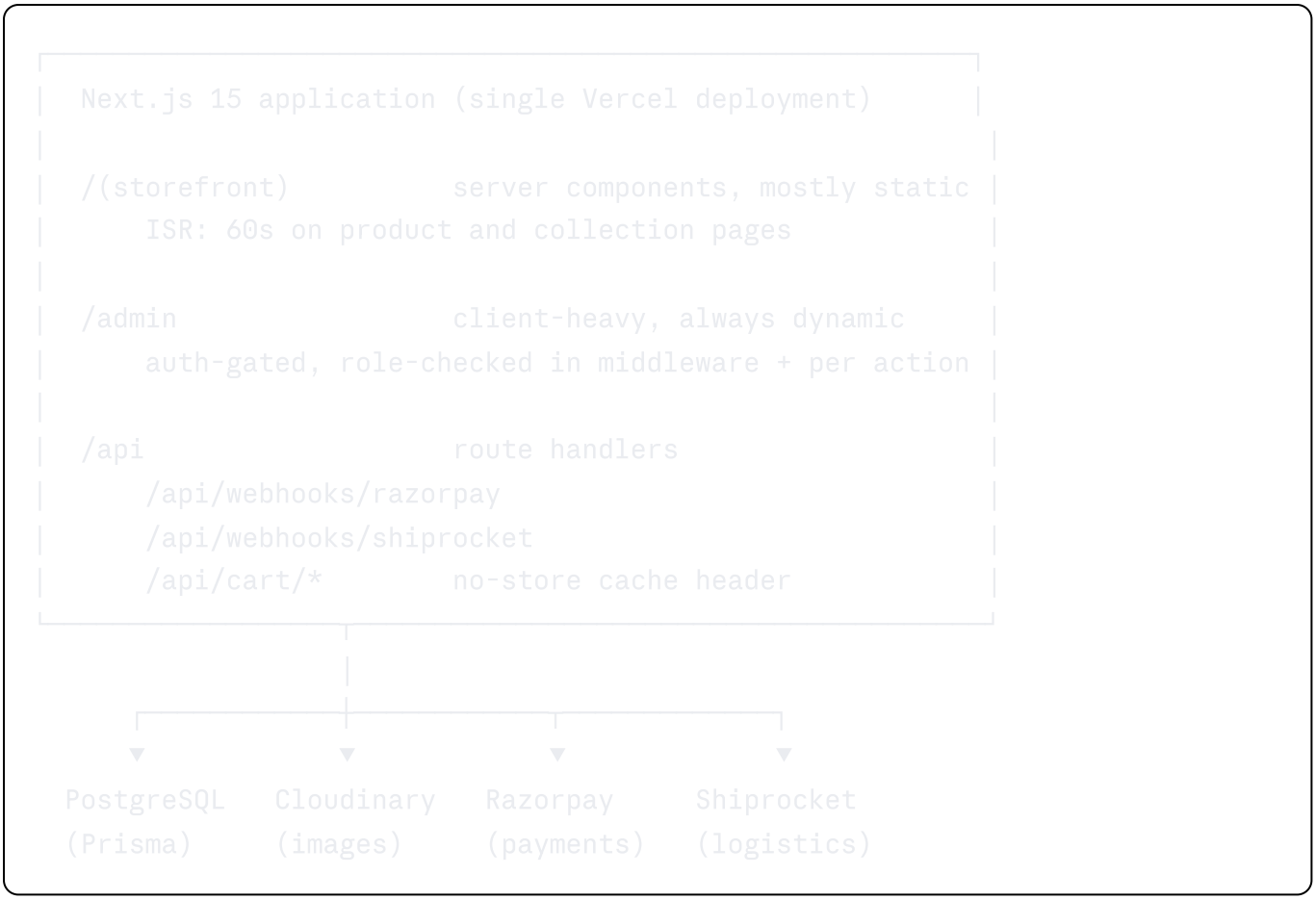
5. Technology stack

Locked. Do not substitute without asking.

Layer	Choice	Reason
Framework	Next.js 15 (App Router) + TypeScript (strict)	Server components keep the storefront fast; one codebase for store and admin
Styling	Tailwind CSS v4	Fast, and the design tokens in §11 map cleanly to it
Admin UI components	shadcn/ui	Accessible primitives, no heavy dependency
Database	PostgreSQL (Neon or Supabase)	Relational data, transactions for orders
ORM	Prisma	Readable schema, good migrations
Validation	Zod	Shared schemas between client and server
Forms	React Hook Form + Zod resolver	
Auth	Auth.js v5	Email+password for staff; phone OTP for customers
Payments	Razorpay	Best Indian coverage: UPI, cards, netbanking, wallets
Image storage	Cloudinary	On-the-fly resizing and WebP/AVIF conversion
Transactional email	Resend	Order confirmations, invoices
Transactional SMS/WhatsApp	MSG91	Order updates — higher open rate than email in India
Shipping	Shiprocket API	Aggregates DTDC, Delhivery, and others; handles COD remittance
PDF invoices	`@react-pdf/renderer`	Server-side invoice generation
Analytics	Vercel Analytics + GA4	
Hosting	Vercel	
Error tracking	Sentry	

Explicitly not using: any headless CMS, Redux, a component library beyond shadcn, or a state manager heavier than Zustand (only for cart).

6. Architecture



Rendering strategy

- Homepage, category pages, product pages: server components with ISR, revalidated on 60s and on-demand via `revalidatePath` when the admin saves a product.
- Cart and checkout: client components, `Cache-Control: no-store`.
- Admin: fully dynamic, no caching.

Money handling

All money is stored as `Int` in **paise**. `₹149.00` is `14900`. Format only at the render boundary with a single shared `formatINR()` helper. There must be exactly one such helper in the codebase.

---

7. Repository structure

```
bbtc-store/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts                  # MX3 range, categories, one combo, one
discount
├── src/
│   ├── app/
│   │   ├── (storefront)/
│   │   │   ├── page.tsx                      # home
│   │   │   ├── shop/page.tsx                 # all products
│   │   │   ├── shop/[category]/page.tsx
│   │   │   ├── product/[slug]/page.tsx
│   │   │   ├── combo/[slug]/page.tsx
│   │   │   ├── cart/page.tsx
│   │   │   ├── checkout/page.tsx
│   │   │   ├── order/[id]/page.tsx            # confirmation + tracking
│   │   │   ├── account/                       # orders, addresses
│   │   │   ├── where-to-buy/page.tsx
│   │   │   ├── about/page.tsx
│   │   │   └── policies/[slug]/page.tsx
│   │   ├── admin/
│   │   │   ├── layout.tsx                     # role gate
│   │   │   ├── page.tsx                       # dashboard
│   │   │   ├── products/
│   │   │   ├── combos/
│   │   │   ├── discounts/
│   │   │   ├── orders/
│   │   │   ├── customers/
│   │   │   ├── inventory/
│   │   │   ├── content/                       # homepage blocks, banners
│   │   │   ├── reports/
│   │   │   └── settings/
│   │   └── api/
│   ├── components/
│   │   ├── ui/                                # shadcn primitives
│   │   ├── storefront/
│   │   └── admin/
│   ├── lib/
│   │   ├── db.ts
│   │   ├── auth.ts
│   │   ├── money.ts                           # formatINR, paise helpers
│   │   ├── gst.ts                             # tax split logic
│   │   ├── pricing.ts                         # THE discount engine — see
```

```
§9
|   |   ├── razorpay.ts
|   |   ├── shiprocket.ts
|   |   └── cloudinary.ts
|   ├── server/
|   |   └── actions/                          # server actions, all Zod-
validated
|   └── types/
├── DECISIONS.md
├── .env.example
└── README.md
```

---

8. Data model

Prisma schema. This is the contract — build to it.

prisma

```
// ---------- Catalogue ----------

model Category {
  id          String    @id @default(cuid())
  name        String                          // "Dish Care"
  slug        String    @unique               // "dish-care"
  description String?
  imageUrl    String?
  sortOrder   Int       @default(0)
  isActive    Boolean   @default(true)
  products    Product[]
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

model Product {
  id            String           @id @default(cuid())
  name          String                               // "MX3 Dish Wash Liquid"
  slug          String           @unique
  brand         String           @default("MX3")
  shortDesc     String                               // one line, used on cards
  description   String           @db.Text            // rich text (HTML), product p
  categoryId    String
  category      Category         @relation(fields: [categoryId], references: [i
  hsnCode       String                               // GST classification, e.g. "3
  gstRate       Int                                  // basis points: 1800 = 18%
  claims        Json             // [{ icon, label }] — the badge row
  ingredients   String?          @db.Text
  usageSteps    Json?            // string[]
  isActive      Boolean          @default(true)
  isFeatured    Boolean          @default(false)
  sortOrder     Int              @default(0)
  variants      ProductVariant[]
  images        ProductImage[]
  reviews       Review[]
  metaTitle     String?
  metaDesc      String?
  createdAt     DateTime         @default(now())
  updatedAt     DateTime         @updatedAt

  @@index([categoryId, isActive])
}

// A variant is a buyable thing. For MX3 this is fragrance x size.
model ProductVariant {
  id             String       @id @default(cuid())
```

```
  productId      String
  product        Product      @relation(fields: [productId], references: [id],
  sku            String       @unique          // "MX3-LEM-500"
  fragrance      String?                       // "Lemon"
  fragranceHex   String?                       // "#F2C14E" — drives the colour
  size           String                        // "500 ml"
  mrpPaise       Int                           // printed MRP
  pricePaise     Int                           // selling price
  wholesalePaise Int?                          // B2B tier, hidden at launch
  weightGrams    Int                           // for shipping rate calculation
  barcode        String?
  isActive       Boolean       @default(true)
  inventory      Inventory[]
  images         ProductImage[]
  createdAt      DateTime      @default(now())
  updatedAt      DateTime      @updatedAt

  @@index([productId, isActive])
}

model ProductImage {
  id        String          @id @default(cuid())
  productId String?
  product   Product?        @relation(fields: [productId], references: [id], on
  variantId String?
  variant   ProductVariant? @relation(fields: [variantId], references: [id], on
  url       String                             // Cloudinary public URL
  publicId  String                             // Cloudinary id, for deletion
  alt       String
  sortOrder Int             @default(0)
}

// ---------- Combos ----------
// A combo is a sellable package of variants at a package price.
// It is NOT a discount rule — it is its own listing with its own page.

model Combo {
  id          String      @id @default(cuid())
  name        String                            // "MX3 Fruit Pack — 3 x 500ml"
  slug        String      @unique
  shortDesc   String
  description String      @db.Text
  imageUrl    String?
  pricePaise  Int                               // the package price
  isActive    Boolean     @default(true)
  isFeatured  Boolean     @default(false)
  items       ComboItem[]
```

```
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
}

model ComboItem {
  id        String         @id @default(cuid())
  comboId   String
  combo     Combo          @relation(fields: [comboId], references: [id], onDel
  variantId String
  variant   ProductVariant @relation(fields: [variantId], references: [id])
  quantity  Int            @default(1)
  // If true, the customer picks which fragrance fills this slot at purchase ti
  isChoice  Boolean        @default(false)
  // When isChoice, restrict the options to these variant ids. Empty = any in t
  choiceOf  String[]
}

// ---------- Inventory ----------

model Location {
  id        String      @id @default(cuid())
  name      String                              // "Palakkad Stock Room"
  address   String
  pincode   String
  isActive  Boolean     @default(true)
  inventory Inventory[]
}

model Inventory {
  id         String         @id @default(cuid())
  variantId  String
  variant    ProductVariant @relation(fields: [variantId], references: [id], on
  locationId String
  location   Location       @relation(fields: [locationId], references: [id])
  quantity   Int            @default(0)
  reserved   Int            @default(0)         // held by pending orders
  lowStockAt Int            @default(10)

  @@unique([variantId, locationId])
}

// ---------- Offers and discounts ----------

enum DiscountType {
  PERCENTAGE          // 10% off
  FIXED_AMOUNT        // ₹50 off
  BUY_X_GET_Y         // buy 2 get 1 free
```

```
  FREE_SHIPPING
  TIERED_QUANTITY     // buy 3+ → 10% off, buy 6+ → 15% off
}

enum DiscountScope {
  ORDER               // applies to cart subtotal
  PRODUCT             // applies to specific products
  CATEGORY            // applies to a category
  COMBO
}

model Discount {
  id              String        @id @default(cuid())
  name            String                            // internal name
  publicLabel     String?                           // shown on the site: "Monsoon
  code            String?       @unique             // null = automatic discount
  type            DiscountType
  scope           DiscountScope
  valuePaise      Int?                              // for FIXED_AMOUNT
  valueBps        Int?                              // for PERCENTAGE, 1000 = 10%
  tiers           Json?                             // for TIERED_QUANTITY: [{minQ
  buyQty          Int?                              // for BUY_X_GET_Y
  getQty          Int?
  minOrderPaise   Int?                              // minimum cart value to quali
  maxDiscountPaise Int?                             // cap on percentage discounts
  targetIds       String[]                          // product/category/combo ids
  usageLimit      Int?                              // total redemptions allowed
  usageCount      Int           @default(0)
  perCustomerLimit Int?
  stackable       Boolean       @default(false)     // can combine with other disc
  priority        Int           @default(0)         // higher wins when not stacka
  startsAt        DateTime
  endsAt          DateTime?
  isActive        Boolean       @default(true)
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt

  @@index([isActive, startsAt, endsAt])
}

// ---------- Customers and orders ----------

enum UserRole { CUSTOMER STAFF MANAGER ADMIN }

model User {
  id              String    @id @default(cuid())
  name            String?
```

```
  email         String?   @unique
  phone         String?   @unique
  passwordHash  String?                          // staff only
  role          UserRole  @default(CUSTOMER)
  isActive      Boolean   @default(true)
  addresses     Address[]
  orders        Order[]
  reviews       Review[]
  createdAt     DateTime  @default(now())
}

model Address {
  id          String  @id @default(cuid())
  userId      String
  user        User    @relation(fields: [userId], references: [id], onDelete: Ca
  fullName    String
  phone       String
  line1       String
  line2       String?
  city        String
  state       String
  pincode     String
  isDefault   Boolean @default(false)
}

enum OrderStatus {
  PENDING_PAYMENT
  CONFIRMED
  PACKED
  SHIPPED
  DELIVERED
  CANCELLED
  RETURNED
  REFUNDED
}

enum PaymentMethod { RAZORPAY COD }
enum PaymentStatus { PENDING PAID FAILED REFUNDED }

model Order {
  id                String        @id @default(cuid())
  orderNumber       String        @unique        // "BB-2026-00001"
  userId            String?
  user              User?         @relation(fields: [userId], references: [id])
  guestEmail        String?
  guestPhone        String?
```

```
  items             OrderItem[]

  subtotalPaise     Int
  discountPaise     Int           @default(0)
  shippingPaise     Int           @default(0)
  codFeePaise       Int           @default(0)
  cgstPaise         Int           @default(0)
  sgstPaise         Int           @default(0)
  igstPaise         Int           @default(0)
  totalPaise        Int

  appliedDiscounts  Json          // snapshot: [{discountId, label, amountPaise

  shippingAddress   Json          // snapshot, never a relation
  billingAddress    Json

  status            OrderStatus   @default(PENDING_PAYMENT)
  paymentMethod     PaymentMethod
  paymentStatus     PaymentStatus @default(PENDING)
  razorpayOrderId   String?
  razorpayPaymentId String?

  shiprocketOrderId String?
  awbNumber         String?
  courierName       String?
  trackingUrl       String?

  invoiceNumber     String?       @unique
  invoiceUrl        String?

  notes             String?
  createdAt         DateTime      @default(now())
  updatedAt         DateTime      @updatedAt

  @@index([status, createdAt])
  @@index([userId])
}

model OrderItem {
  id             String   @id @default(cuid())
  orderId        String
  order          Order    @relation(fields: [orderId], references: [id], onDele
  // Snapshot fields — the order must stay correct even if the product is later
  variantId      String?
  comboId        String?
  nameSnapshot   String
  skuSnapshot    String
```

```
  imageSnapshot  String?
  quantity       Int
  unitPricePaise Int
  totalPaise     Int
  hsnCode        String
  gstRate        Int
}

model Review {
  id         String   @id @default(cuid())
  productId  String
  product    Product  @relation(fields: [productId], references: [id], onDelete
  userId     String?
  user       User?    @relation(fields: [userId], references: [id])
  authorName String
  rating     Int                                  // 1-5
  title      String?
  body       String   @db.Text
  isApproved Boolean  @default(false)             // moderation required
  createdAt  DateTime @default(now())

  @@index([productId, isApproved])
}

// ---------- Editable content ----------

model ContentBlock {
  id        String   @id @default(cuid())
  key       String   @unique      // "home_hero", "announcement_bar", "home_sto
  data      Json                  // shape depends on key, validated by Zod per
  isActive  Boolean  @default(true)
  updatedAt DateTime @updatedAt
}

model Retailer {
  id       String  @id @default(cuid())
  name     String
  address  String
  city     String
  district String
  pincode  String
  phone    String?
  lat      Float?
  lng      Float?
  isActive Boolean @default(true)
}
```

```
model AuditLog {
  id         String   @id @default(cuid())
  userId     String
  action     String                  // "product.update"
  entityType String
  entityId   String
  before     Json?
  after      Json?
  createdAt  DateTime @default(now())

  @@index([entityType, entityId])
}
```

Seed data required (`prisma/seed.ts`): four categories; the MX3 product with six fragrance variants in 500 ml; one combo ("MX3 Fruit Pack — 3 x 500 ml") with a `isChoice` slot; one active percentage discount; two locations; one ADMIN user.

---

9. The pricing engine

This is the single most important module in the codebase. Put it in `src/lib/pricing.ts` and unit test it before wiring any UI to it.

Contract

ts

```ts
type CartLine = {
  kind: 'variant' | 'combo'
  id: string
  quantity: number
  unitPricePaise: number
  productId?: string
  categoryId?: string
}

type PriceResult = {
  lines: Array<CartLine & { lineDiscountPaise: number; finalTotalPaise: number
  subtotalPaise: number
  discountPaise: number
  appliedDiscounts: Array<{ id: string; label: string; amountPaise: number }>
  shippingPaise: number
  codFeePaise: number
  taxBreakdown: { cgst: number; sgst: number; igst: number }
  totalPaise: number
}

function calculateCart(input: {
  lines: CartLine[]
  discountCode?: string
  shippingState?: string
  paymentMethod: 'RAZORPAY' | 'COD'
}): PriceResult
```

Rules, applied in this exact order:

1. Compute line subtotals from `unitPricePaise * quantity`.
2. Gather all candidate discounts: every active automatic discount within its date window, plus the coupon matching `discountCode` if supplied and valid.
3. Filter out any discount whose `minOrderPaise` exceeds the subtotal, whose `usageLimit` is exhausted, or whose `perCustomerLimit` is reached for this customer.
4. Apply **product-scoped** and **category-scoped** discounts to matching lines first.
5. Then apply **order-scoped** discounts to the remaining subtotal.
6. If two applicable discounts both have `stackable = false`, keep only the one with the higher `priority`; break ties by larger rupee value to the customer.
7. Cap every percentage discount at `maxDiscountPaise` when set.
8. Discount can never exceed the line or order subtotal. Never produce a negative total.
9. Compute shipping (§12), then the COD fee if applicable.
10. Compute GST last, on the **post-discount** amount, per §12.

Combo pricing: a combo's `pricePaise` is the price. Do not compute it from its components. The admin sets it directly, and the product page shows the component sum struck through so the saving is visible.

Test cases that must pass:

- Empty cart returns zero everywhere without throwing.
- A 10% discount capped at ₹100 on a ₹2,000 cart yields exactly ₹100 off.
- A non-stackable 15% and a non-stackable ₹200-off never both apply.
- Kerala shipping address produces CGST + SGST; a Tamil Nadu address produces IGST.
- A tiered quantity discount picks the correct tier at the boundary (exactly 3, exactly 6).
- Rounding: every result is an integer number of paise, and line discounts sum exactly to `discountPaise` with no off-by-one drift.

---

10. Storefront specification

10.1 Global

- **Announcement bar** — one line, editable via `ContentBlock` key `announcement_bar`, dismissible, remembers dismissal for the session. Must never contain a hardcoded offer.
- **Header** — logo left; category links (Dish Care / Home Care / Car Care / Fabric Care); Where to Buy; search; account; cart with item count. Sticky on scroll, condensed after 80px.
- **Mobile nav** — full-screen drawer. Categories expand in place, no nested drill-down.
- **Footer** — three office addresses, toll free number, policy links, social links, email capture. GST number displayed (required for a manufacturer selling direct).
- **Cart drawer** — slides from the right on add-to-cart. Shows line items, applied offers by name, subtotal, and progress toward the free-shipping threshold.

10.2 Homepage

Sections in order. Every section's copy and imagery is admin-editable.

1. **Hero** — single full-bleed product image on a flat fragrance-coloured background, one headline, one subline, one button. No carousel. Carousels reduce conversion and the current site has one.
2. **Claim badge row** — four claims with icons: 2X grease removal, gentle on hands, uses less water, refreshing fragrance. Pulled from `Product.claims`.
3. **The fragrance rail** — the signature element. Six MX3 fragrances as a horizontal scroller; hovering or tapping one shifts the section background to that fragrance's `fragranceHex`. This is the one piece of motion on the page.
4. **Category tiles** — four tiles, real photography, no placeholder images ever.
5. **Combo strip** — featured combos with the saving shown in rupees, not percent. "Save ₹90" outperforms "Save 15%" in this category.

6. **Founder story** — short, first person, with the 2019 origin. Photograph of the founder, not a stock image.
7. **Reviews** — approved reviews only, with product name attached.
8. **Where to buy teaser** — links to the retailer list.

10.3 Product page

- Image gallery, first image large; thumbnails below on mobile, left rail on desktop.
- Product name, short description, star rating linking to reviews.
- **Fragrance selector** — swatches using `fragranceHex`. Selecting one updates the price, the images, the URL query, and the accent colour of the page. Out-of-stock fragrances are shown but disabled with the reason stated.
- Size selector where more than one size exists.
- Price block: selling price, MRP struck through, "You save ₹X (Y%)".
- Quantity stepper, then Add to cart. Sticky add-to-cart bar on mobile.
- Delivery check: pincode input → serviceability, ETA, and whether COD is available.
- Accordion: Description, How to use, Ingredients, Shipping and returns.
- "Frequently bought together" — surfaces relevant combos.
- Reviews with a write-review form (goes to moderation).

10.4 Combo page

Same as product page, plus: the component list with each item's individual price, the component sum struck through, and the package price. Where a `ComboItem` has `isChoice`, render a fragrance picker per slot with clear labels ("Bottle 1", "Bottle 2", "Bottle 3").

10.5 Cart and checkout

Checkout is **one page, three collapsible steps** — not a multi-page wizard.

1. **Contact** — phone number required, email optional. Phone OTP for returning customers.
2. **Delivery** — saved addresses for logged-in users; pincode auto-fills city and state.
3. **Payment** — Razorpay or COD. COD shows its fee if any, and is hidden entirely for non-serviceable pincodes.

Order summary is always visible: desktop as a right rail, mobile as a collapsible top bar. Every discount is listed by its `publicLabel` so the customer sees why the price dropped.

Guest checkout must work. Do not require account creation.

10.6 Post-purchase

- Confirmation page with order number and expected delivery window.
- Email (Resend) and SMS/WhatsApp (MSG91) on: order confirmed, shipped with AWB, delivered.
- Order tracking page keyed on order number + phone, no login required.
- GST invoice PDF downloadable once the order is confirmed.

10.7 Other pages

`/where-to-buy` (searchable retailer list by district and pincode), `/about`, `/policies/[slug]` for shipping, returns, privacy, terms.

---

11. Design system

The brief is a Kerala manufacturer of fruit-scented dish wash. The distinctive material in that world is **the fragrance range** — six fruits, six colours, one bottle. That is the design system. The site is chromatically neutral until a fragrance enters the frame, and then it takes over the accent layer.

The company name is Bright Blue. The base is therefore blue, but a *deep, ink-like* blue rather than the cheerful mid-blue every detergent brand uses. The brightness comes from the fruit, not the chrome.

11.1 Colour tokens

```css
--bb-ink:        #0B1F3A;   /* deep navy — text, header, footer */
--bb-ink-soft:   #3C4F6B;   /* secondary text */
--bb-paper:      #FBFAF7;   /* page background — warm off-white, not pure white
--bb-line:       #E3E0D8;   /* hairline borders */
--bb-signal:     #0F9CD6;   /* the one "bright blue" — CTAs, links, focus rings
--bb-success:    #2E7D5B;
--bb-warning:    #B4501C;
```

Fragrance accents, stored per variant in `fragranceHex`:

```
Lemon        #F2C14E
Peach        #F09A76
Blueberry    #6B6BC4
Apple        #7BAE5C
Grapes       #9B6BA8
Green Mango  #A8BF4A
```

Discipline rule: exactly one fragrance colour is active on screen at a time. It appears in the product image background, the selected swatch, and nothing else. Never tint text, never tint the CTA — the CTA is always `--bb-signal`. This keeps the page from turning into a fruit salad and makes the fragrance switch feel like an event.

11.2 Typography

Role	Face	Source	Use
Display	**Clash Display**	Fontshare (free)	Headlines, product names. Medium and Semibold only.
Body	**Satoshi**	Fontshare (free)	Everything else. Regular, Medium, Bold.
Utility	**JetBrains Mono**	Google Fonts	SKUs, order numbers, prices in the admin panel

Both Fontshare faces are free for commercial use and self-hosted via `next/font/local`. Do not use Inter — it is the default that makes every AI-built site look identical.

Type scale (1.25 ratio, fluid via `clamp`):

```
display-lg   clamp(2.5rem, 6vw, 4.5rem)   Clash Display Semibold, -0.02em
display-sm   clamp(1.75rem, 4vw, 2.5rem)  Clash Display Medium, -0.01em
heading      1.5rem                        Clash Display Medium
body-lg      1.125rem                      Satoshi Regular, 1.6 line-height
body         1rem                          Satoshi Regular, 1.6
caption      0.875rem                      Satoshi Medium, 0.02em tracking,
uppercase
```

11.3 Layout and components

- 12-column grid, 1280px max content width, 24px gutters mobile / 40px desktop.
- Spacing scale: 4, 8, 12, 16, 24, 32, 48, 64, 96.
- Border radius: 4px on inputs and buttons, 16px on cards, full on swatches. Nothing else.
- Shadows: one elevation only, `0 1px 3px rgba(11,31,58,0.08)`. Resist adding more.
- Buttons: primary is solid `--bb-signal` with white text; secondary is a 1px `--bb-ink` outline; there is no third button style.

11.4 Photography rules

Enforce these in the admin panel with an upload checklist:

- Every product shot is a single bottle, centred, on a flat colour field matching its fragrance hex. No lifestyle clutter behind the bottle.
- Square 1:1 for cards, 4:5 for the product page hero.
- Minimum 2000px on the long edge before Cloudinary processing.
- No stock photography anywhere on product or category pages. The current site uses stock hands and a bicycle image — none of that survives this rebuild.

11.5 Motion

Total motion budget for the storefront: the fragrance rail transition (240ms ease-out on background-color), cart drawer slide (200ms), and hover lift on cards (120ms). Everything respects `prefers-reduced-motion`. No scroll-jacking, no parallax, no page-load animation.

11.6 Copy voice

Plain, confident, specific. State what the product does, do not sell it. Sentence case everywhere. **No emoji anywhere on the storefront** — the current product descriptions are emoji-dense and it reads as amateur for a manufacturer. Buttons name their action and keep that name through the flow: "Add to cart" produces "Added to cart".

---

12. India-specific requirements

12.1 GST

- Each product carries an `hsnCode` and `gstRate`. Dish wash liquid is typically HSN 3402 at 18% — **confirm with BBTC's accountant before launch, do not assume.**
- Place of supply determines the split. Seller state is Kerala.
  - Delivery to Kerala → CGST (rate/2) + SGST (rate/2)
  - Delivery to any other state → IGST (full rate)
- Prices displayed to customers are **inclusive of GST**. The tax is back-calculated for the invoice: `taxable = inclusive * 10000 / (10000 + gstRate)`.
- Tax is computed on the post-discount amount.
- Invoice must show: seller GSTIN, invoice number and date, HSN per line, taxable value, tax rate, tax amount split, total in figures and words, and place of supply.
- Invoice numbers are sequential per financial year with no gaps. Generate them inside the same database transaction that confirms the order.

12.2 Payments

- Razorpay Orders API. Create a Razorpay order server-side, never trust client amounts.
- **Verify the webhook signature on every callback.** Order status changes only from the webhook, never from the client-side success handler.
- Support UPI, cards, netbanking, wallets — all default in Razorpay Checkout.
- COD: configurable flat fee, a minimum and maximum order value, and a pincode allowlist or blocklist managed in the admin.

12.3 Shipping

- Shiprocket API for rates, label generation, AWB assignment, and tracking.
- Serviceability check by pincode on the product page and at checkout.
- Rate logic: free shipping above an admin-set threshold; otherwise weight-based slabs computed from `ProductVariant.weightGrams`.
- Fall back gracefully: if Shiprocket is unreachable at checkout, use the admin's flat-rate table rather than blocking the order.

12.4 Communications

- Order updates via MSG91 on WhatsApp with SMS fallback. In this market WhatsApp is read; email often is not.
- Email is used for the invoice PDF and marketing consent.

---

13. Non-functional requirements

Area	Requirement
Performance	LCP < 2.5s, INP < 200ms, CLS < 0.1 on a mid-range Android on 4G. Lighthouse mobile ⩾ 90.
Images	All served via Cloudinary as AVIF/WebP with explicit width and height. No layout shift.
SEO	Server-rendered product pages, `Product` and `Offer` JSON-LD, canonical URLs, XML sitemap, per-product meta from `metaTitle`/`metaDesc`.
Accessibility	WCAG 2.1 AA. Visible focus rings, keyboard-navigable swatches and drawers, form errors linked to inputs, 4.5:1 text contrast.
Security	Server-side role checks on every mutation; Zod validation on every input; rate limiting on OTP, login, and coupon endpoints; secrets only in env; no PII in logs; webhook signature verification.
Reliability	Order creation is a single database transaction. Payment webhooks are idempotent by `razorpayPaymentId`.
Auditability	Every price, product, and discount change writes an `AuditLog` row.
Browser support	Last 2 versions of Chrome, Safari, Firefox, Edge; Android Chrome; iOS Safari 15+.

---

14. Build phases

Build in this order. Do not start a phase before the previous one meets its criteria.

Phase 0 — Foundation

Scaffold Next.js 15 + TypeScript strict + Tailwind v4. Configure Prisma against Neon. Add Auth.js with the five roles. Set up Cloudinary, Sentry, and the `.env.example`. Implement `lib/money.ts` and `lib/gst.ts` with tests. **Done when:** `pnpm dev` runs, a seeded ADMIN user can log in, `/admin` is role-gated, and money and GST helpers pass their unit tests.

Phase 1 — Catalogue and admin CRUD

Full Prisma schema and migrations. Seed script. Admin screens for categories, products, variants, and image upload with reordering. **Done when:** a manager can create the

complete MX3 range with six fragrance variants, images, prices, and HSN codes entirely through the UI, with no direct database access.

Phase 2 — Storefront browse

Homepage, category pages, product page with the fragrance selector, search. Design system tokens and fonts implemented. ISR configured with on-demand revalidation on admin save. **Done when:** the MX3 range is browsable, the fragrance rail works, Lighthouse mobile ⩾ 90, and no placeholder or stock image appears anywhere.

Phase 3 — Cart and pricing engine

Zustand cart with persistence. `lib/pricing.ts` complete with the full test suite from §9. Cart drawer and cart page. **Done when:** every test case in §9 passes and the cart totals match hand calculations for five worked examples.

Phase 4 — Combos and discounts

Combo model, combo builder in the admin (including `isChoice` slots), combo pages. Discount admin covering all five `DiscountType` values, with a preview showing the effect on a sample cart before the manager saves. **Done when:** a manager can build a 3-bottle mixed-fragrance combo and launch a tiered-quantity discount without help, and both price correctly in the cart.

Phase 5 — Checkout, payments, orders

One-page checkout. Razorpay integration with webhook verification. COD with pincode rules. Order creation transaction, order number generation, admin order management. **Done when:** a real ₹1 Razorpay test transaction completes end to end, the webhook moves the order to CONFIRMED, and inventory decrements correctly.

Phase 6 — Fulfilment, invoices, comms

Shiprocket integration, AWB and tracking, GST invoice PDF, email and WhatsApp notifications, customer order tracking page. **Done when:** an order can be shipped from the admin, the customer receives WhatsApp and email updates, and the invoice PDF is GST-compliant per §12.1.

Phase 7 — Content, reviews, polish

Editable homepage blocks, retailer locator, reviews with moderation, policy pages, accessibility audit, SEO audit, error and empty states. **Done when:** the WCAG AA audit passes and every empty state gives the user a next action.

Phase 8 — Launch

Data migration from WooCommerce, 301 redirects from old URLs, GA4 and Search Console, staff training, load test, go-live checklist.

---

15. Environment variables

```
DATABASE_URL=
DIRECT_URL=

AUTH_SECRET=
AUTH_URL=

CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=

SHIPROCKET_EMAIL=
SHIPROCKET_PASSWORD=

RESEND_API_KEY=
MSG91_AUTH_KEY=
MSG91_WHATSAPP_TEMPLATE_ID=

SENTRY_DSN=

NEXT_PUBLIC_SITE_URL=
NEXT_PUBLIC_GA_ID=

BBTC_GSTIN=
BBTC_LEGAL_NAME=
BBTC_INVOICE_PREFIX=BB
```

---

16. Definition of done for v1

- [ ] A manager adds a new product, prices it, uploads images, and publishes — no developer
- [ ] A manager builds a combo with a customer-chosen fragrance slot
- [ ] A manager launches a tiered quantity discount with a preview before saving
- [ ] A customer completes a Razorpay purchase on mobile in under 90 seconds
- [ ] A customer completes a COD purchase and receives WhatsApp confirmation
- [ ] A GST invoice PDF is generated and matches the accountant's requirements
- [ ] Lighthouse mobile ⩾ 90 on home, category, and product pages
- [ ] WCAG 2.1 AA audit passes
- [ ] Zero placeholder images, zero lorem ipsum, zero emoji on the storefront
- [ ] All old brightblue.in URLs 301 to their new equivalents

---

17. Open questions for BBTC

Answer these before Phase 1. They change the schema.

1. **HSN code and GST rate** for dish wash liquid — confirm with the accountant.
2. **Sizes** — is MX3 sold in more than 500 ml? List every size per fragrance.
3. **Actual prices** — MRP and selling price per SKU. The current site shows ₹0.00 everywhere.
4. **Which entity sells online?** BBTC, BBM, or BBC? This determines the GSTIN on the invoice.
5. **Full product range** — beyond MX3, what car care, fabric care, and home care SKUs exist?
6. **Combos to launch with** — which packs, at what price?
7. **Shipping policy** — free shipping threshold, and COD fee if any.
8. **Return policy** — window, condition, and who pays return shipping.
9. **B2B** — is a distributor login needed at launch or later?
10. **Product photography** — does it exist at 2000px+, or does it need to be shot?

---

18. Risks

Risk	Impact	Mitigation
Thin margins on a low-ticket FMCG item make paid shipping unviable	High	Model unit economics before setting the free-shipping threshold; push combos hard to raise AOV
Product photography does not exist at usable quality	High	Budget a one-day shoot before Phase 2; the design depends on clean bottle shots
GST configured incorrectly	High	Accountant sign-off on one live invoice before launch
COD return-to-origin rate	Medium	Start COD on a pincode allowlist; expand as data comes in
Scope creep into subscriptions or B2B mid-build	Medium	Both are explicitly non-goals in §3; log requests, do not build them
Admin panel too complex for staff	Medium	Usability test Phase 4 with the actual person who will use it